High-Scale Analytics Project
Text Classification & Dynamic Time Warping

Introduction

This is a complete report for the project of high scale analytics of DSIT. We start with **Exercise 1**, which examines a large dataset (over 100,000 rows). We then perform *text preprocessing* (going from basic cleaning to GPU-accelerated spaCy) and compare common ML methods (SVM, Random Forest, k-NN).

In **Exercise 2**, we speed up **k-Nearest Neighbor** ($k = 7$) using **Locality Sensitive Hashing (LSH)**. MinHash helps us avoid comparing documents that are clearly different.

Finally, in **Exercise 3**, we compute DTW distances for time series and discuss why time-warping can be helpful.

---

1 Exercise 1: Data Import, Exploration, and Preprocessing

1.1 Data Import and Initial Exploration

We have a news dataset with many samples, each labeled with a category, there are 4 of them. We first look at the dataset size, structure, memory usage, possible GPU acceleration, and whether we need to sample. We also check the column types and any missing values before starting text preprocessing.

1.1.1 Data Loading and Structure

```
import pandas as pd
data = pd.read_csv("train.csv")
```

Columns are:

- `Id`: integer ID
- `Title`: short headline
- `Content`: main text
- `Label`: a category (Business, Entertainment, Technology, Health)

`data.head()` confirms each record has a title, content, and numeric ID.

1.1.2 Dataset Size, Missing Values, and Class Distribution

`data.isnull().sum()` shows no missing values; we also check if any category is significantly underrepresented. If one class is much larger, models might overpredict it resulting in overfit. As seen in the figure below there is indeed an imbalance in our dataset. For now, we will not deal with it, however, we keep that in mind in case our evaluation is not satisfying, we may consider augmentation technniques.

Entertainment	44834
Technology	30107
Business	24834
Health	12020

1.2 Data Preprocessing Techniques

1.2.1 Merging Title and Content

```
data["Text"] = data["Title"] + " " + data["Content"]
```

By merging title and content we make sure that we don't have any information loss. Considering title and content at once makes the processing of the data and the training of the models way easier.

1.2.2 Text Cleaning Steps

We perform:

1. **Lowercasing**: Convert all text to lowercase so that words like "Apple" and "apple" are treated the same. This helps avoid confusion and keeps things consistent.

2. **Punctuation Removal**: Take out punctuation marks like commas, periods, and question marks because they don't usually affect the meaning of the text. This makes the text simpler and easier to work with.

3. **Stopword Removal**: Remove common words like "the," "is," and "and" that don't add much meaning. This helps focus on the important words in the text.

4. **Stemming or Lemmatization**: Simplify words to their root form, like turning "running" into "run." This groups similar words together, making it easier to understand the main ideas.

5. **Removing Numbers**: Get rid of numbers, since they usually don't help us understand the text. This keeps the focus on the actual words.

6. **Removing Extra Whitespace**: Clean up extra spaces between words or lines to make the text look neat and tidy. This avoids any confusion when reading or processing the text.

1.3 Text Preprocessing Pipeline

We apply incrementally its processing step for each of the two models on a subset of our data. We want to evaluate the impact of its step.

1.3.1 Illustrative Results: Stemming Pipeline

Step	SVM Accuracy	RF Accuracy
Remove Punctuation	0.618	0.740
Remove Stopwords	0.684	0.770
Apply Stemming	0.716	0.768
Remove Numbers	0.718	0.776
Remove Extra Whitespace	0.718	0.768

Analysis of Stemming Results:

- Removing punctuation/stopwords produces the largest boost since these tokens add noise.
- Stemming improves SVM more than RF.
- Numbers removal yield additional, though minor, gains.
- Removing extra whitespace does not improve accuracy, in fact it slight decreases it in RF. We believe that it is actually random, since extra white spaces are actually giving no more information.

1.3.2 Illustrative Results: Lemmatization Pipeline

Step	SVM Accuracy	RF Accuracy
Remove Punctuation	0.618	0.736
Remove Stopwords	0.684	0.768
Apply Lemmatization	0.716	0.766
Remove Numbers	0.718	0.778
Remove Extra Whitespace	0.718	0.770

Analysis of Lemmatization Results:

- Lemmatization converts words to their base form, enhancing interpretability.
- Accuracy improvements are similar to stemming; the choice depends on specific domain needs.
- RF still outperforms SVM.

1.4 GPU-Accelerated Preprocessing with spaCy and Hugging Face

For large-scale datasets, CPU loops are slow. Using `spacy.require_gpu()` with Hugging Face `Dataset` enables batch processing on the GPU, significantly reducing preprocessing time.

1.5 Model Training and Hyperparameter Tuning

We split the data (80/20 or 90/10), encode labels, and then tune hyperparameters.

1.5.1 Saving/Loading Preprocessed Data

```
data.to_pickle("processed_data.pkl")
data = pd.read_pickle("processed_data.pkl")
```

We store the pre-processed datasets so that we do not need to repeat heavy preprocessing when tweaking models.

1.5.2 Hyperparameter Tuning with Optuna

For hyperparameter tuning we will use ooptuna to get advantage of its pruning methods and the ability to better converge to the best hyperparameter configuration. We tune:

- Vectorizer parameters: `ngram_range`, `min_df`, `max_df`, etc.
- SVM parameters: `C`, `kernel`, `class_weight`.
- RF parameters: `n_estimators`, `max_depth`, `min_samples_split`, etc.

We perform 5-fold CV for each trial on a subset for 50 trials to determine the best settings.

1.5.3 Best Parameters and Final Steps

```
\subsection{Storing Best Hyperparameters}
\label{sec:bestparams}
\begin{lstlisting}
best_svm_params = {
    'ngram_range': '(1, 4)',
    'max_features': 10000,
    'binary': False,
    'min_df': 2,
    'max_df': 0.9,
    'strip_accents': 'unicode',
    'C': 0.01012588590893926,
    'kernel': 'linear',
    'class_weight': None
}

best_rf_params = {
    'ngram_range': '(1, 4)',
    'max_features': None,
    'binary': True,
    'min_df': 1,
    'max_df': 0.95,
    'strip_accents': None,
    'n_estimators': 613,
    'max_depth': 30,
    'min_samples_split': 8,
    'min_samples_leaf': 4,
    'max_features_rf': 'sqrt',
    'bootstrap': False,
    'class_weight': 'balanced'
}
```

Best SVM Parameters: The best parameters for the SVM retrieved by optuna are:

- **ngram_range=(1,4):** Captures unigrams to 4-grams, learning both individual words and short phrases. This range balances capturing context with not inflating the feature space excessively.
- **max_features=10000:** Limits the vocabulary size to the 10,000 most frequent tokens. A large corpus can yield a huge vocabulary; selecting the top tokens reduces noise and improves efficiency.
- **binary=False:** Uses raw term counts rather than binary presence. This allows the SVM to weigh words by frequency, which is particularly important for high-dimensional, sparse data.

- **min_df=2** and **max_df=0.9:** Filter out extremely rare and overly common tokens. Rare tokens add noise and may overfit, while overly common tokens do not contribute to discrimination.
- **strip_accents='unicode':** Normalizes accented characters. This ensures that accented and unaccented versions of the same word are not treated as different, which is important in multilingual contexts. Even we have removed punctuation in the pre-processing step we still enable this feature in cases anything was missed.
- $C = 0.01012588590893926$**:** A very small penalty parameter. The data is highly sparse and noisy; a small C encourages a wide margin and prevents overfitting. Larger C values tended to overfit to noise.
- **kernel='linear':** Uses a linear kernel. Linear kernels are computationally efficient and effective for high-dimensional text data.
- **class_weight=None:** No reweighting of classes is applied. Even with the class imbalance, the hyperparameter tuning resulted in no need for weight in classes.

Best RF Parameters: The best parameters for the RF retrieved by optuna are:

- **ngram_range=(1,4):** Maintained for consistency with SVM, capturing both individual words and short phrases.
- **max_features=None:** Uses all filtered features. RF can handle a large feature space when other parameters constrain tree complexity.
- **binary=True:** Converts raw counts to binary presence/absence. This simplifies splits in tree-based models.
- **min_df=1** and **max_df=0.95:** Retains rare tokens while removing near-universal ones. Although retaining very rare tokens can be noisy, the ensemble nature of RF mitigates this risk.
- **n_estimators=613:** The number of trees used in the forest. Hyperparameter tuning indicated that around 613 trees strike a balance between performance and computation.
- **max_depth=30:** Limits the depth of each tree. This prevents overfitting in the high-dimensional feature space.
- **min_samples_split=8** and **min_samples_leaf=4:** Ensure that splits and leaves have a minimum number of samples. These settings avoid splitting on very small subsets that could capture noise.
- **max_features_rf='sqrt':** Uses the square root of the total number of features at each split. This standard practice reduces correlation among trees and improves generalization.
- **bootstrap=False:** Disables bootstrapping. In this high-dimensional text scenario, not using bootstrap improved performance .
- **class_weight='balanced':** Adjusts class weights according to frequency. This prevents the majority class from dominating the splits, which is important given slight class imbalances. Interestingly, the best hyperaprameter for SVM was none class weight, things have changed for RF.

1.6 GPU-Accelerated ML with cuML

Libraries `cuML` (from NVIDIA RAPIDS) offer GPU-based SVM and Random Forest implementations, which significantly reduce training time on large, high-dimensional datasets—a critical advantage for our corpus.

1.7 Exercise 1: Observed Results & Analysis

We now return to our full datase and perform 5-fold CV for the best hyperapraerameters for both models.

- **Final Accuracy:**
  - SVM: Mean Accuracy: 0.9669, Std Dev: 0.0013.
  - RF: Mean Accuracy: 0.6887, Std Dev: 0.0057.

  Analysis: The high accuracy of SVM is attributable to its effective handling of sparse data with a linear kernel and a low C. The low C value prevents the model from overfitting to noise inherent in the data.

- **Impact of Preprocessing:**
  - Removing punctuation and stopwords yields the largest improvement by eliminating non-informative tokens.
  - Stemming and lemmatization provide additional gains and help reduce feature redundancy.

- **Model Comparison:**
  - SVM outperforms RF because its margin-based approach is better suited for the sparse, high-dimensional data; RF tends to overfit without aggressive tuning.

1.8 DeBERTa

Finally, we also fine-tune a transformers based model for classification, DeBERTa. We will not provide the details here since this is not actually part of the project, however we provide the code in the notebook and some comments on the parameters and training loss we have chosen. Actually, it outperfors both in validation and test set all different methods, but by a really small margin while it requires extra time bot at training and inference time.

2 Exercise 2: Nearest Neighbor Search with Locality Sensitive Hashing

2.1 Objective

k-NN can achieve high accuracy, but its brute-force implementation has a complexity of $O(N \times M)$, which is computationally expensive for large datasets, where N is the number of training documents and M is the number of test documents. LSH (Locality Sensitive Hashing) is used to reduce the number of comparisons by focusing only on documents that are likely to be similar. This allows to scale k-NN to much larger datasets.

Comparison of k-NN vs. SVM/RF:

- k-NN can sometimes match or exceed the accuracy of SVM/RF on small subsets, but its computational cost increases rapidly with dataset size. In our case, we had to consider a vectorized implementation of k-NN using libraries like NumPy, otherwise the Kaggle notebook was running out of RAM. Even with vectorization, the brute-force approach becomes prohibitively slow for large N.

- In contrast, SVM and RF are pre-trained and offer faster prediction times, making them more suitable for deployment scenarios where speed is critical. However, training them can be computationally expensive, especially for very large datasets. They also may not achieve the same level of accuracy as k-NN in some specific applications.

2.2 k-NN with Jaccard Similarity (Brute Force)

1. **Tokenization:** Tokenize documents into sets of words or n-grams.
2. **Jaccard Similarity Computation:** Compute the Jaccard similarity for each test document against all training documents. The Jaccard similarity between two sets A and B is defined as: $J(A, B) = \frac{|A \cap B|}{|A \cup B|}$.
3. **Neighbor Selection:** Select the top $k = 7$ neighbors based on the highest Jaccard similarity scores.
4. **Label Prediction:** Determine the label for the test document by majority vote among its k nearest neighbors.

Brute-force k-NN is highly accurate (96.5% accuracy in the validation set we created) but becomes unscalable as dataset size grows. The $O(N \times M)$ complexity makes it impractical for large datasets.

2.3 LSH-Based k-NN Methods

MinHash LSH Concept: MinHash LSH offers an efficient way to approximate the Jaccard similarity and significantly speed up k-NN search. The core idea is to hash documents into buckets such that similar documents have a high probability of colliding in the same bucket.

1. **MinHash Signature Generation:**
   (a) Create a random permutation of the rows (words/n-grams) in the vocabulary.
   (b) For each document, create a MinHash signature by finding the first row index where each word/n-gram in the document appears in the permuted order. Repeat this process multiple times with different random permutations to create a signature vector. Each element in the signature vector corresponds to one permutation.

   The MinHash signature approximates the Jaccard similarity between documents. The probability of two documents having the same MinHash value for a given permutation is equal to their Jaccard similarity.
2. **Banding:** Divide the MinHash signature into b bands, each consisting of r rows. This creates a total of b hash tables.
3. **Hashing into Buckets:** Hash each band of each document's MinHash signature into a bucket within its corresponding hash table. Documents with identical bands will collide in the same bucket.
4. **Candidate Pair Generation:** Documents that hash to the same bucket in at least one band are considered candidate pairs for further similarity calculation. This step reduces the number of pairwise comparisons.
5. **Exact Similarity Calculation :** For candidate pairs, compute the exact Jaccard similarity (or another similarity measure) to refine the neighbor selection. This step is only performed on a small subset of the data, dramatically improving performance.
6. **Neighbor Selection and Label Prediction:** Select the top k neighbors from the candidate pairs based on their exact similarity scores and predict the label by majority vote.

2.4 Parameter Justification and Result Analysis

2.4.1 Observed Results:

- **Brute-Force-Jaccard**
  Results:

Table 1: Performance Comparison of k-NN Implementations

Type	Build Time (s)	Query Time (s)	Total Time (s)	Fraction Overlap	Accuracy
LSH-Jaccard-16	104.88	766.49	871.38	0.2075	0.8349
LSH-Jaccard-32	108.53	77.75	186.27	0.2846	0.8843
LSH-Jaccard-64	115.12	127.35	242.47	0.4062	0.9321
Brute-Force-Jaccard	0.00	155.32	155.32	1.0000	0.9650
LSH-Jaccard-Forest-16	100.21	25.58	125.79	0.0477	0.6236
LSH-Jaccard-Forest-32	107.14	28.51	135.64	0.1055	0.7257
LSH-Jaccard-Forest-64	115.02	33.76	148.78	0.1006	0.7205
LSH-Jaccard-Vectorized-16	83.47	1119.03	1202.50	0.2088	0.8346
LSH-Jaccard-Vectorized-32	93.75	105.13	198.88	0.2871	0.8805
LSH-Jaccard-Vectorized-64	102.50	205.15	307.65	0.4114	0.9298

  - *Build Time*: 0s (no preprocessing).
  - *Query Time*: 155.32s (moderate for exact search).
  - *Accuracy*: 96.5% (highest, as expected).

  Computes exact Jaccard similarities for all pairs. Slow for large datasets but unavoidable for higher accuracy. The moderate query time here suggests the dataset is small enough that brute-force remains practical.

- **LSH-Jaccard-16/32/64**
  Standard LSH with MinHash .
  Results:
  - *Build Time*: Increases with permutations (104.88s → 115.12s) due to longer MinHash signatures.
  - *Query Time*: Non-linear trend (766s → 78s → 127s).
  - *Accuracy*: Improves with permutations (83.5% → 93.2%).

  Why:
  - At 16 permutations (likely 8 bands × 2 rows), high false positives force lengthy candidate verification (766s).
  - At 32 permutations (e.g., 16 bands × 2 rows), better band/row balance reduces false positives (78s query).
  - At 64 permutations (e.g., 32 bands × 2 rows), excessive bands increase hash table lookups, slowing queries (127s).

  Accuracy improves with permutations because more hash functions better approximate true Jaccard similarity.

- **LSH-Jaccard-Forest-16/32/64**
  LSH Forest supports building an index without specifying a similarity threshold, enabling retrieval of top-k neighbors directly.
  Results:
  - *Build Time*: Slightly higher than standard LSH (100s → 115s).
  - *Query Time*: Fastest overall (25.58s–33.76s).
  - *Accuracy*: Lowest (62.4%–72.6%).

  Why:
  - LSH Forest uses multiple prefix trees to index data. This allows for efficient retrieval of top-k similar items without pre-defined thresholds.
  - The fast query times are due to the efficient organization of the data in the forest structure.
  - The lower accuracy suggests that the forest's structure, while fast, leads to some loss of precision in identifying the most similar items. This is due to the inherent approximations of LSH and the specific tree structure used. Since the first approach after retrieving the neighbors also performs full Jaccard similarity, jsut getting the top 7 neigbors from the forst is way faster but worse in accuracy.

- **LSH-Jaccard-Vectorized-16/32/64**
  LSH with vectorized MinHash computation.
  Results:
  - *Build Time*: 10–15% faster than standard LSH due to batch processing.
  - *Query Time*: Slower than non-vectorized (1119s–205s vs. 77s–127s).
  - *Accuracy*: Matches standard LSH for equivalent permutations.

  Why:
  - Vectorization speeds up MinHash generation (lower build time) but introduces overhead in query phase.
  - For 16 permutations, vectorization overhead (memory alignment, data reshaping) dominates benefits.
  - At 32 permutations, reduced overhead improves query time (105s) but remains slower than standard LSH (78s).
  - At 64 permutations, larger signatures stress memory bandwidth, negating vectorization gains.

- **Cross-Method Observations**:
  - **Build Time**: Scales with permutation count for all LSH variants. Vectorized builds are 10–15% faster.
  - **Query Time**: Forests are 3–30x faster than standard LSH but with reduced accuracy.
  - **Accuracy**: Brute-force > Standard LSH > Forest.
  - **Parameter Sensitivity**: Optimal permutations vary by method:
    * Standard LSH: 32 permutations (best speed/accuracy balance).
    * Forest: 16 permutations (fastest queries).
    * Vectorized: 32 permutations (least bad query time).

2.5 LSH Error Analysis

The $b < 2$ error occurs when no candidates share buckets in any band. This is more likely with: - High similarity thresholds (overly selective bands). - Sparse data (few overlapping hash buckets). - Insufficient permutations (limited collision chances).

Method-Specific Error Behavior:

- **Standard LSH**: Prone to this error at 16 permutations (low collision probability).
- **Forest**: Avoids the error by design (no thresholding) but replaces it with low accuracy.
- **Vectorized**: Inherits standard LSH's error behavior but with no mitigation.

Important Notes:

- This error is worse for sparse, high-dimensional data.
- The forest method avoids the error but gives very inaccurate results.

3 Exercise 3: Dynamic Time Warping (DTW)

3.1 Overview

DTW measures the similarity between two numeric sequences even if their time axes are misaligned. It is particularly useful in applications such as speech recognition and signature matching.

Comparisons with Other Distance Metrics:

- Euclidean Distance is simpler but does not account for time shifts.
- DTW warps the time axis to align similar patterns, providing a more accurate similarity measure when sequences are out of phase.

3.2 Euclidean Distance and DTW Matrix

```
def euclidean_distance(p1, p2):
    return np.sqrt((p1 - p2) ** 2)

def dynamic_time_warping(seq_a, seq_b):
    matrix = np.zeros((len(seq_a) + 1, len(seq_b) + 1))
    matrix[0,:] = np.inf
    matrix[:,0] = np.inf
    matrix[0,0] = 0
    for i, v1 in enumerate(seq_a):
        for j, v2 in enumerate(seq_b):
            cost = euclidean_distance(v1, v2)
            matrix[i+1, j+1] = cost + min(
                matrix[i, j+1],
                matrix[i+1, j],
                matrix[i, j]
            )
    return matrix[-1,-1]
```

DTW has a complexity of $O(L^2)$ for sequences of length L. The whole procedure takes approximately 22 minutes in kaggle cpu.

3.3 Observed Results:

- A **low DTW distance** indicates that the sequences align well after warping, suggesting similar underlying patterns.
- A **high DTW distance** indicates significant differences in shape or phase.
- Processing 1002 pairs in 22 minutes demonstrates that while DTW provides a realistic similarity measure, its quadratic complexity can be a bottleneck for very long or numerous sequences.
- In cases where sequences are nearly in phase, Euclidean distance might be preferred for its speed.

4 Conclusion

Key Findings From Each Exercise

- **Exercise 1 (Text Classification)**
  - **SVM** achieved approximately 96–97% accuracy, largely due to the use of a linear kernel and a low C (0.01). *Explanation:* The low C was necessary because the data is highly sparse; a small C maintains a wide margin, reducing overfitting. Larger C values led to memorization of noise.
  - **Random Forest** reached around 70–80% accuracy. *Explanation:* Although RF can capture non-linearities, the high dimensionality makes it prone to overfitting without extensive tuning.
  - Preprocessing (e.g., stopword removal, lemmatization/stemming) significantly boosted performance by removing noise and reducing feature redundancy.
  - **k-NN** works well on small datasets but becomes impractical at scale.
- **Exercise 2 (Nearest Neighbor & LSH)**
  - Brute-force **k-NN** finds exact neighbors and achieves high accuracy in classification. For this dataset it is not computationally prohibitive but it can become in larger scale.
  - **S**ome LSH variant provide significant speedup in query times (around 40.62% with 64 permutations).

- **Exercise 3 (Dynamic Time Warping)**
  - **DTW** aligns time-shifted sequences more effectively than Euclidean distance. *Explanation:* DTW warps the time axis to match similar patterns, which is particularly useful when sequences are out of phase.
  - The quadratic time complexity ($O(L^2)$) can be a bottleneck for very long sequences or large datasets.

Overall Observations and Lessons

- **Parameter Selection:**
  - SVM's low C (0.01) and linear kernel are optimal for our high-dimensional, sparse text data.
  - The chosen `ngram_range` of (1,4) captures both single words and short phrases without exploding the feature space.
  - RF's constraints (`max_depth=30`, `min_samples_split`=8, `min_samples_leaf`=4) help mitigate overfitting, although RF still underperformed compared to SVM.
- **Model Comparison:**
  - Margin-based classifiers like SVM excel in handling sparse, high-dimensional text data compared to tree-based methods.
- **Time-Series Analysis:**
  - DTW is advantageous when time shifts are present, although its quadratic complexity may necessitate further optimization.
- **GPU Acceleration:**
  - Utilizing GPU-enabled libraries (spaCy with GPU, cuML) is essential for handling large-scale, high-dimensional data efficiently.